6.1220 CRIB SHEET

Master Theorem: $T(n) = aT(\frac{n}{b}) + \Theta(n^c)$

Condition	Solution
$c < \log_b a$	$T(n) = \Theta(n^{\log_b a})$
$c = \log_b a$	$T(n) = \Theta(n^c \log n)$
$c > \log_b a$	$T(n) = \Theta(n^c)$

Runtime Recurrence: (induction)
$T(n) = 2T(\frac{n}{7}) + T(\frac{2n}{3}) + O(n)$ $T(n) = O(n)$
Guess $T(k) \le ck$ for $k < n$, prove $T(n) \le cn$.
$T(n) \le 2c\frac{n}{7} + c(\frac{2n}{3}) + c_1 n = n(\frac{20}{21}c + c_1)$
Pick $c_1 = \max(T(1), 21c_2)$
for base case $T(1)$ to hold
note to prove Θ: $T(n) = \Omega(n)$
prove both O & Ω! since level 1 already takes $O(n)$ time.

P & NP: P → Polynomial time NP → verified in $O(n^c)$ time
NP-Complete → $\in$ NP, possible to reduce to NP
NP-Hard → at least as hard as NP-complete problems
→ NP-Complete problem Y reducible to NP hard problem X

Rank-Finding / Median Finding

Given S, find x in S s.t. rank(x) = i (for median, $i = \frac{n+1}{2}$)
Divide into $\frac{n}{5}$ groups, Sort each and find median,
recursively find median x of the $\frac{n}{5}$ group medians.
Compute $L = \{y \in S \mid y < x\}$, $G = \{y \in S \mid y > x\}$, rank(x) = |L| + 1
If rank(x) = i, return x.
If rank(x) < i, recurse on L.
If rank(x) > i, recurse on G with i - rank(x).
Claim: x is 7/10-balanced ($\max(|L|, |G|) \le 7/10n$)
Runtime: $T(n) = T(\frac{7n}{10}) + T(\frac{n}{5}) + O(n) = O(n)$

Recurrence: $T(n) = T(\frac{n}{5}) + T(\frac{7n}{10}) + O(n)$
# of groups $n - (3 \cdot \frac{1}{2} \cdot \frac{n}{5})$
median in groups of 5

Union find:

- make-set(x): add $\{x\}$ to collection
- find-set(x): return REP[S(x)]
- union(x, y): $S(x) \cup S(y)$

Implementation: forest of trees (Union-by-rank (merging smaller tree into larger tree) + Path compression (find(x), sets parent of x to be root))
Runtime: Inverse-Ackermann $O(\alpha(n))$ amortized

Amortized Cost:

Cost = T if k operations have a cumulative cost of $\le kT$.

Potential Method: Define potential function Φ, maps DS to number

- $\hat{c}_i = c_i + \Delta\Phi_i$ AND $\Phi_i > \Phi_0$ $\forall i \to \sum \hat{c}_i \ge \sum c_i$ Required: $\Phi_i > \Phi_0$, $\Phi_0 = 0$
- $\hat{c}_i \le k \cdot c_i^{OPT}$ $\forall i \to \sum \hat{c}_i \le k \cdot \sum c_i^{OPT}$

Online Algorithm: length undetermined, algo. read piece by piece, make decision for each
A deterministic algorithm is **α-competitive** if
$\forall R, C_A(R) \le \alpha C_{OPT}(R) + C$, $C_{OPT}(R)$ is cost for optimal.
want potential method to be a measure of some kind of distance between D_i & D_i^{OPT}

Randomized Algorithms: (vs deterministic)

Uses random #'s to make decisions, diff. outputs & runtime
Monte Carlo: Good runtime, sometimes right
Las Vegas: Always right, sometimes bad runtime.

Paranoid Quick Select: (Las Vegas, rank finding)
- pick random pivot
- compute L, G
- If rank(x) < n/4 or > 3n/4, pick another pivot. Else, recurse on L or G.

Runtime: $O(n)$ $O(n \log n)$ if $\frac{c}{c}$-balanced c constant < 1

Paranoid Quicksort: ← median of $\frac{n}{2}$ sized groups
- pick pivot 3/4 balanced, quicksort L & G

Runtime: $T(\frac{3n}{4}) + T(\frac{n}{4}) + O(n) = O(n \log n)$
median $\ge \frac{n}{4}$ elems. → $\frac{3}{4}$-balanced

Hashing

Hash functions: $h: \mathcal{U} \to \{0, \dots, m-1\}$ ($\mathcal{U}$ = universe of possible keys)
Hash n items into m slots. (m = # of keys)
Supports: insert(k,v), delete(k), search(k)

Simple Uniform Hashing Assumption (SUHA):
$P(h(k) = h(k')) \le \frac{1}{m}$ for $k \ne k'$ (fixed function good for random inputs)

Universal Hashing Family (UHF): (probability of being hashed to same key is low)
Collection H of functions h s.t. $\forall k \ne k' \in \mathcal{U}$:
$P(h(k) = h(k')) \le \frac{1}{m}$, $h \in H$ (for each of the hash function in the UHF)
Given k, k', we can randomly pick h and condition holds.
$\mathbb{E}[\#\text{collisions}] \le 1 + \frac{n-1}{m} < 1 + \alpha$ (load function, $\alpha = \frac{n}{m}$)

Open Addressing: (probing sequence until empty slot)
When collision: find another slot in the table.
$\mathbb{E}[\#\text{probes to insert}] \le \frac{1}{1-\alpha}$
→ linear probing $h(k,i) = [h'(k) + i] \bmod m$
→ double hashing $h(k,i) = [h_1(k) + i h_2(k)] \bmod m$ ← not UHA

Universal Hashing Assumption (UHA):
probe sequence of random keys
↕
random permutation of $0, \dots, m-1$
collision $\le \frac{1}{m}$

Static Dictionary: Given $k_1, \dots, k_n \in \mathcal{U}$ up front.
$O(n)$ pre-processing time
$O(n)$ space
$O(1)$-time SEARCH in worst case
(only supports SEARCH)

Perfect Hashing: (for static dictionary)

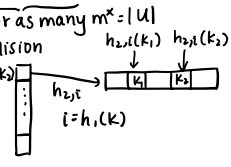
Two Level Hashing / FKS Algorithm
1. $h_1: \mathcal{U} \to \{0, \dots, m-1\}$ from a UHF with $m = n$. Hash all n keys $\{k_1, \dots, k_n\}$ with chaining using h. $m = n$, n_j items in bin j
2. Each bin has hash $m_j = n_j^2$ Prob $\ge \frac{1}{2}$

FKS Algorithm:
1. try random 1st level h with $m = n$ until $\sum n_j^2 \le 4n$
2. for each j try random h_j with $m = n_j^2$ till no collision

$O(n)$ build, $O(1)$ search, $O(n)$ space

$\binom{n}{k} = \frac{n!}{k!(n-k)!}$

want prime m:
Base m rep of keys $k = (k^{(2)}, k^{(1)}, k^{(0)})$
x for as many $m^x = |\mathcal{U}|$ collision



Minimum Spanning Tree (MST)

unique edge weights → unique MST

Subtree with $E = |V| - 1$ that connects all V and have min. weight.
- $\forall e$, there is a unique path $P_e \in T$ connecting u & v with $P_e \cup \{e\}$ forming a fundamental cycle.
- Cut & Swap property: for every $e \notin T$ & any $e' \in P_e \in T$, Can create spanning tree $T' = T \cup \{e\} \setminus \{e'\}$

A cut of G is a partition of V into $(S, V \setminus S)$; any edge (x,y) crosses cut if $x \in S, y \in V \setminus S$; a cut respects a set of edges.
If no edge of A crosses the cut, an edge is a light edge if it has min. wt. among all edges crossing cut.

Strong safe edge statement: Let A be a proper subset of edges, and $(S, V \setminus S)$ be a cut that respects A, light edge e is safe for A.

Meta-Greedy MST (G(V,E)):
A = {}
//Invariant: A contains subset of edges in some MST
while A not a spanning tree:
- find some "safe" e s.t. $A \cup \{e\}$ remains invariant
- $A \leftarrow A \cup \{e\}$

Return A

If edge wts sorted: $O(m\alpha(n))$

dense → $|E| = O(|V|^2)$

Prim's Algo.: $O(|E| + |V| \log |V|)$
find lowest wt. edge s.t. $u \in T$, $v \notin T$, add to tree.

Kruskal's Algo.: forest-of-trees
pick min. wt. edge that connects 2 different trees.
$O(|E| \log |E|) + O(|E| \alpha(|V|))$
$O(m \log m) + O(m\alpha(n))$

$O(m + n \log n)$

Probability

Union-bound: events $A_1, A_2, \dots, A_n$, $\Pr[\text{union } A_i \forall i] \le \sum \Pr[A_i]$
Linearity of Expectations: $\mathbb{E}[\sum X_i] = \sum \mathbb{E}[X_i]$
Chebyshev Inequality: $\mu = \mathbb{E}[X]$, $\sigma^2 = \text{Var}[X]$, $\Pr[|x - \mu| \ge k] \le \frac{\sigma^2}{k^2}$ ($k > 0$)
Markov's Inequality: $\Pr[X \ge k] \le \frac{\mathbb{E}[X]}{k}$, x nonnegative, $k > 0$
Chernoff's Bounds: Random binomial variable $X = B(n,p)$ ($\sum$ of independent Bernoulli variables)
$\mu = \mathbb{E}[X]$

$\text{Var}(X) = \mathbb{E}[(x - \mathbb{E}(X))^2] = \mathbb{E}[x^2] - (\mathbb{E}(X))^2$

- $\Pr[X \ge (1+\beta)\mu] \le e^{-\frac{\beta^2\mu}{3}}$, for $0 < \beta \le 1$
- $\Pr[X \ge (1+\beta)\mu] \le e^{-\frac{\beta\mu}{3}}$, for $\beta \ge 1$
- $\Pr[X \le (1-\beta)\mu] \le e^{-\frac{\beta^2\mu}{2}}$, for $0 < \beta < 1$

Maximum Flow

$G(V,E,s,t,c)$ m edges n vertices

- Feasibility $f(u,v) \le c(u,v)$
- Flow conservation $\sum_u f(u,v) = 0$
- Skew symmetry $f(u,v) = -f(v,u)$
- Max Flow Min Cut:
  $|f| = C(S)$ for some s-t cut S
  $f = F^*$
  f admits no augmenting path in G_f

Flow decomposition lemma: Subgraph of G with nonzero flow edges can be decomposed into flow paths & cycles.

Capacity of cut: $C(S) = \sum_{u \in S} \sum_{v \in V \setminus S} c_f(u,v)$ → If there ∃ a s-t cut with $C(S) \neq 0$, then max flow is nonzero

Cut Properties: for any s-t cut S, S' and flow f → $f(S) \le C(S)$, $f(S) = f(S')$, $|f| = f(S)$

Residual Network $G_f(V, E_f, s, t, c_f)$

- for all pairs (u,v): Only adding edge if not using all of capacity
  $(u,v) \in E_f \Leftrightarrow c_f(u,v) = c(u,v) - f(u,v) > 0$
- for every $(u,v) \in E_f$:
  its weight is $c_f(u,v) = c(u,v) - f(u,v)$

Augmenting Path
Any path from s to t in G_f is an augmenting path in G with respect to f.
The flow value can be increased along an augmenting path p by:
$c_f(p) = \min_{(u,v) \in p} \{c_f(u,v)\}$ ← Residual bottleneck capacity

Algorithms for finding max flow:

Ford Fulkerson

- Start with 0 flow
- Find augmenting path p in G_f push $c_f(p)$ along p, repeating until no augmenting flows left

Runtime: $O(|F^*||E|) = O(mnC)$
O(m) per iteration
push ≥ 1 flow per iter
C: max capacity of an edge

Flow integrality theorem: If capacities are integers, then F^* is integral. FF will find an integral max flow.

Edmonds-Karps

- Find shortest augmenting path (BFS)
- push flow.

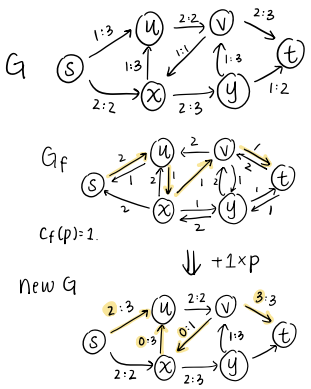
Runtime: $O(m^2 n)$

Maximum Bottleneck Algo

(push most flow)

- Find max residual bottleneck capacity path (binary search c)
- Push flow

Runtime: $O(m^2 \log(n) \log(nC))$



Game Theory

Choosing deterministically (no randomness) ← Randomness in choice (probability of choosing something)

Players, actions, action profile, pure strategy, mixed strategy, utility, payoff matrix M

Best response: Best utility given S_i (all other players' strategy)

Dominant strategy: S_i is the best response $\forall S_i$

If there ∃ a best response (dominant strategy), there ∃ a pure best response.

Every finite game has a potentially mixed nash equilibrium (mutual best response)

If one action completely dominates the other, can remove from payoff matrix to calculate equilibrium. ← probabilities for picking choices that result in same payoff regardless of what opponent picks.

Nash equilibrium: no deviation can provide a strictly better utility.

Strategies for pure nash equilibrium doesn't correlate to strategies for mixed.

2-Player Zero Sum Game: always ∃ nash equilibrium → Assume P_1 action: P_2's utility based on that action. Assume P_2 action: P_1's utility based on that action

Min-Max Theorem: $\max_{S_1} \min_{S_2} S_2^T M S_1 = \min_{S_2} \max_{S_1} S_1^T M S_2$

maximizing one's self utility, minimizing opponent's utility

Intractability

P, NP, etc. only contains decision problems

difficulty (can be same)

Decision	Search	Optimization
Yes or No	find < k	min/max shortest/longest

P: Solvable in polynomial time Runtime $O(n^c)$ for $n = |x|$
∃ deterministic algo. $A_\pi(x)$ s.t. $A_\pi(x) = \text{YES} \Leftrightarrow \pi(x) = \text{YES}$

NP: ∃ deterministic verifier V_π s.t.
$\pi(x) = \text{YES} \Leftrightarrow \exists y\ V_\pi(x,y) = \text{YES}$
x: instance y: certificate

CoNP: ∃ deterministic verifier V_π s.t.
$\pi(x) = \text{NO} \Leftrightarrow \exists y\ V_\pi(x,y) = \text{YES}$
All NP problems reducible to an NP-complete problem.

π: NP-hard if $\forall Q \in NP$, $Q \le_p \pi$. (π at least as hard as Q)
Q: NP-Complete IFF $Q \in NP$ & Q ∈ NP-hard

Reduction: $Q \le_p \pi$ (instance of Q, instance of π)
Deterministic algorithm R: $R(x) = y$
R runs in polynomial time $Q(x) = \text{YES} \Leftrightarrow \pi(y) = \text{YES}$
If we can solve π, we can solve Q.

NP-Complete Proof:
1) Prove NP with polynomial time in certificate and instance
2) a. Describe reduction (polynomial time & size in input)
   b. Show $Q(x) = \text{YES} \Leftrightarrow \pi(y) = \text{YES}$.

Clique is NP-Complete Proof:
1) Clique ∈ NP: Verifier can interpret the proof as a list of vertices and outputs YES IFF
   (1) There ∃ ≥k vertices.
   (2) Every pair of distinct vertices is connected by an edge in the graph.
   Can be checked in poly-time, size of proof poly.
2) Clique is NP-Hard: Vertex Cover $\le_p$ Clique
   (1) (G,k) vertex cover → complement graph G'=(V,E') s.t. $E' = \{(u,v): (u,v) \notin E\}$ $O(|V|^2)$
   G', k = n-k → instance of clique
   (2) Vertex-cover = YES ⇔ Clique = YES

Problems to know:

- **Circuit-SAT**: Boolean circuit C on $x_1, \dots, x_n$, can C = 1? NP, NP-hard, NP-complete
- **3-SAT**: F: (− OR − OR −) ∧ (− OR − OR −) Is F satisfiable? NP-complete
- **Vertex Cover**: Given G, k, ∀ edge, one or more vertex in S, $|S| \le k$. NP-complete
- **Clique**: undirected G, is there ≥ k vertices that form a clique? NP-complete
- **Subset-Sum**: set of n integers and t, ∃ A ⊆ S s.t. $\sum a = t$? NP-complete
- **Knapsack**: n objects (v_i, c_i), ∃ subset A s.t. $\sum_A v \ge V$ and $\sum_A c \le C$? NP-Complete
- **3-Color & K-Color**: NP-complete
- **2-Color** ∈ P (bipartite graph) (BFS + alternate color) 3-color $\le_p$ K-color $G \to (G, 3)$

Linear Programming

$\max \vec{c}^T \vec{x}$ subject to $A\vec{x} \le \vec{b}$, $\vec{x} \ge \vec{0}$ (A: m×n constraint matrix)

$\min \vec{b}^T \vec{y}$ subject to $A^T \vec{y} \ge \vec{c}$, $\vec{y} \ge \vec{0}$

Primal to Dual LP:

- primal constraints → dual variables
  $A_{j,1} x_1 + \dots + A_{j,n} x_n \{\le, \ge, =\} b_j \to y_j$
  $\le$: $y_j \ge 0$ $\ge$: $y_j \le 0$ $=$: y_j unrestricted
- primal variables → dual constraints
  $A_{1,i} y_1 + \dots + A_{m,i} y_m \{\le, \ge, =\} c_i \to x_i$
  primal: max, min
  x_i unrestricted: = =
  $x_i \ge 0$: ≥ ≤
  $x_i \le 0$: ≤ ≥

For each number ∈ R: $x = x^+ - x^-$, $x^+ \ge 0$, $x^- \ge 0$

Primal/Dual	Bounded	Unbounded	Infeasible
Bounded	✓	×	✓
Unbounded	×	×	✓
Infeasible	✓	✓	✓

Feasibility Region: all possible solutions (non-convex)

Weaker redundant: Removing degenerate constraint doesn't affect optimal solution.

If optimal value is finite: LP bounded

Duality: Let $\vec{x}, \vec{y}$ be primal & dual solutions
Weak duality → then $c^T \vec{x} \le b^T \vec{y}$
(In standard form:) primal objective value ≤ dual objective value
Strong duality → primal and dual have equal optimal solution if either bounded (holds if both primal and dual are feasible)

Complementary slackness: If dual/primal constraint not tight (never equality), then corresponding primal/dual variable = 0.

if one infeasible, other has to be either infeasible OR unbounded

Taking dual of LP example:

$\max\ x - y + 2z$
subject to $-x + 3y \le -3$
$x + 3z \ge 2$
$y + z = 7$
$x, y \ge 0$

dual variables
$a \ge 0$
$b \le 0$
c unrestricted

$\min\ -3a + 2b + 7c$
$x \to -a + b + 0 \cdot c \ge 1$
$y \to 3a + 0b + c \ge -1$
$z \to 0a + 3b + c = 2$

$\min\ -3a + 2b + 7c$
subject to $-a + b \ge 1$
$3a + c \ge -1$
$3b + c = 2$
$a \ge 0, b \le 0$

LPs either have:
- exactly 1 solution
- infinitely many solutions
- no solution

Approximation Algorithms

(smaller α that A achieves → better of an algorithm A is)

A is an α-approximation algorithm for p (minimization) IFF $\forall p \in P, \frac{A(p)}{opt(p)} \le \alpha \; ; \alpha \ge 1$ $[A(p) \le \alpha OPT(p)]$

A is an α-approximation algorithm for p (maximation) IFF $\forall p \in P, \frac{opt(p)}{A(p)} \le \alpha \; ; \alpha \ge 1$ $[A(p) \ge \frac{1}{\alpha} OPT(p)]$

Maximum Matching:
u,v can match if $(u,v) \in E$ and not already matched.
M is a valid match if all vertex degree ≤ 1.

Greedy 2-Approx:
M={}, loop until M doesn't change:
if $e \in E \setminus M$ s.t. $M \cup \{e\}$ is valid matching, set $M = M \cup \{e\}$

Minimum Vertex Cover:
take endpoints of maximum matching and output to V, greedy 2-Approx.

Multiplicative Weights

n experts, each give advice, choose what to do
want to: minimize regret = $M^T - \min M_i^T$
Define $m_i^t = 0$ if expert i is correct a time t, else 1
$M_i^T = \sum_{t=0}^{T} m_i^t$ and m^t, M^T for us.

1.) Perfect expert: $E^T = \{$expert w/ no mistakes$\}$
follow majority advice. if wrong, majority wrong.
$\ge 1/2$ remaining experts eliminated
total # of mistakes $\le \log_2 n$

2) Weighted Majority Algorithm:
Parameter: $\varepsilon \in (0, \frac{1}{2}]$
Initialization: $w_i^1 \leftarrow 1$, $\forall i = 1, \dots, n$
On each day $t = 1, \dots, T$:
- compare total weights of experts predicting 0 and total weights of experts predicting 1.
  $\sum_{i:\pi_i^t=0} w_i^t$ vs. $\sum_{i:\pi_i^t=1} w_i^t$
- $d^t = 0$ or 1 depending on which group has more
- After binary event realized, update weights:
  $w_i^{t+1} \leftarrow \begin{cases} w_i^t & \text{if } \pi_i^t = b^t \\ w_i^t(1-\varepsilon) & \text{if } \pi_i^t \ne b^t \end{cases}$

3) Multiplicative Weights Update Algorithm:
Parameter: $\varepsilon \in (0, \frac{1}{2}]$
Initialization: $w_i^1 \leftarrow 1$ $\forall$ $i = 1, \dots, n$
On each day $t = 1, \dots, T$:
- $d^t = \pi_i^t$ w/ prob. $p_i^t = w_i^t / \Sigma_i w_i^t$ } choose expert i with prob. p_i^t
- After event realized, update weights.
  $w_i^{t+1} \leftarrow \begin{cases} w_i^t & \text{if } \pi_i^t = b^t \\ w_i^t(1-\varepsilon) & \text{if } \pi_i^t \ne b^t \end{cases}$

Theorem: The predictions of above algorithm satisfy:
$\sum_{t=1}^{T} \mathbb{E}_{d^t \sim p^t}[\mathbb{1}_{d^t \ne b^t}] \le (1+\varepsilon) \min_i \sum_{t=1}^{T} \mathbb{1}_{\pi_i^t \ne b^t} + \frac{\log(n)}{\varepsilon}$
(expected # of wrong predictions)

Sublinear Algorithms

Types of Approximation:

- "Classical" Approximation (optimization)
  Approximate diameter of a point set.
  Algo: Pick random point k.
  Pick point ℓ to maximize $D_{k\ell}$ → output it. $\varepsilon \ge d^*/2$
- Property Testing (decision-YES/NO)
  Return YES if output is YES, NO if prob. $\ge 3/4$ input is "far" from YES else: YES/NO for input "close" to YES instance.

↳ Testing "connectedness"
Repeat for $\frac{c}{\varepsilon d}$ times ($c \ge 3$):
Pick random node s, and run BFS from s until:
→ either $\ge \frac{2}{\varepsilon d}$ distinct vertices visited (large CC)
→ or realize that s lies in the connected component of size $< \frac{2}{\varepsilon d}$ nodes and outputs NO.
If nothing fails, outputs YES.
$\Pr[\text{no iteration picked small CC}] \le \frac{1}{4}$
Note: $-\ln(\frac{1}{x}) = \ln(x)$

↳ Testing "sortedness"
- List is ε-close to be sorted if you can delete $< \varepsilon n$ elements and still be sorted.
- $c > 2$, Repeat $\frac{c}{\varepsilon}$ times:
  Pick a random $i \in \{0, \dots, n-1\}$
  Binary search for x_i in L
  If search finds inconsistency or doesn't end up at i: output NO
  If never failed → output YES

for each query: $\Pr[\text{picking missing edge in CC}] \ge \varepsilon$ since graph is ε-far from complete. (not ε-close)
$\Pr[\text{not finding any missing edges}] \le (1-\varepsilon)^m$
$(1-\varepsilon)^m \le e^{-\varepsilon m}$ m: # of queries
$-\varepsilon m \le \ln(\delta)$ δ: desired error $\varepsilon m \ge \ln(\delta)$

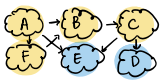
Markov Chains & Random Walks

also called "walk matrix"

Markov chain is time-homogenous if there is a transition probability matrix W s.t. $\forall t \in \mathbb{N}$ and pairs of states u,v:
$\Pr[X_{t+1} = v \mid X_t = u] = W(u,v)$
↳ Every time-homogenous Markov chain has a stationary distribution.
If strongly connected, is unique.

Connectivity:
2 vertices u,v are connected if there $\exists$ a directed path from u to v and vice versa. ($u \rightsquigarrow v$)
↝ is an equivalence relation
↳ Reflexive $u \rightsquigarrow u$ $\forall u \in V$
↳ Symmetric $u \rightsquigarrow v$ IFF $v \rightsquigarrow u$
↳ Transitive $u \rightsquigarrow v$ and $v \rightsquigarrow w \to u \rightsquigarrow w$

V can be partitioned into equivalence classes → strongly connected components (SCCs)
Lemma: The "induced" directed graph on SCCs is acyclic.
Component is transient if it has an outgoing edge. Otherwise, recurrent.



Cycles: (usually proved w/ self loops)
The period of a vertex is the greatest common divisor of all lengths of all directed cycles in its SCC.
Vertex is periodic if period > 1. Otherwise, aperiodic. (undirected graph → vertex period ≤ 2)
Lemma: An undirected connected graph has a period vertex IFF it is bipartite. (no odd-length cycles)

Fundamental Theorem of Markov Chains:
strongly connected + aperiodic → every random walk converges to a unique stationary distribution.

Metropolis-Hastings:
$g(e) = g(x,y)$ → "proposal distribution"
probability of choosing eage (x,y) from curr node x.
Assuming all edges equally likely:
$g(x,y) = \frac{1}{\#\text{ of outgoing edges of } x}$

$W(x,y) = g(x,y) \cdot P_{accept}(x,y)$
$P_{accept}(x,y) = \min\left(1, \frac{\pi_y \cdot g(y,x)}{\pi_x \cdot g(x,y)}\right)$
move from $x \to y$ w/ probability $P_{accept}(x,y)$ ↑ otherwise, stay at x.

Markov Chains

future state of system depends on current state but not previous states nor time t.

Transition Probability from state i to state j: w_{ij}
→ sum going out = 1, sum coming in can be anything.

Transition Matrix:
$W = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ w_{21} & \ddots & & w_{2n} \\ \vdots & & \ddots & \vdots \\ w_{m1} & w_{m2} & \cdots & w_{mn} \end{bmatrix}$
$P(X_{n+1} = i) = \sum_{j=1}^{m} v_i w_{ij}$
$v_j = P(X_n = j)$

equilibrium: incoming = outgoing for a vertex u
$\pi_u \cdot P(u \to v) = \pi_v \cdot P(v \to u)$
transition prob. from $u \to v$.

Detailed balance: MC reversible if $\exists$ distribution s.t. (π is stationary) $\pi_u w_{uv} = \pi_v w_{vu}$ $\forall$ u,v (same mass transfer)

Stationary Distribution: $\vec{\pi} W = \vec{\pi}$, $\Sigma \pi_i = 1$

Period of vertex v = GCD of directed cycle lengths from v

W: a: $[a \to a \;\; a \to b \;\; a \to c]$, b: $[b \to a \;\; b \to b \;\; b \to c]$, c: $[c \to a \;\; c \to b \;\; c \to c]$
(π_a π_b π_c)

$\vec{\pi} = [\pi_a \; \pi_b \; \pi_c]$
$\vec{\pi} W = \vec{\pi}$ (solving for $\vec{\pi}$)
$\pi_a = w_{aa} \cdot \pi_a + w_{ba} \cdot \pi_b + w_{ca} \cdot \pi_c$
$\pi_b = w_{ab} \cdot \pi_a + w_{bb} \cdot \pi_b + w_{cb} \cdot \pi_c$
$\pi_c = w_{ac} \cdot \pi_a + w_{bc} \cdot \pi_b + w_{ca} \cdot \pi_c$

Markov Chain Monte Carlo (MCMC): · Metropolis-Hastings · Gibbs Sampling
Distribution V, know V up to normalization → $\vec{V}$
sample from V without finding distribution.
MC from $\vec{V}$ & proposed distribution g
$W(u,v) = g(u|v) P_{acc}(v,u)$
$P_{acc}(v,u) = \min\{1, v_y g(x|y) / v_x g(y|x)\}$
In the long run, MC looks like $\vec{\pi}$. Mixing time: expected time until MC converges

Sketching & Streaming

$\log_2(\text{num})$ = # of bits to encode num

Streaming: can only keep limited data, $< n$
Sketching: input $c \to c(x)$ sketch of smaller size s.t. compute $f'(x)$ using c that approximates $f(x)$
Quality: f' is (ε, δ)-approximate of f if $f'(x) \in ((1-\delta)f(x), (1+\delta)f(x))$
Want θ, have estimates θ' s.t. $\mathbb{E}[\theta'] = \theta$, $\text{var}[\theta'] = \sigma^2$,
get independent $\theta_1', \theta_2', \dots, \theta_n'$
1) Mean: Return $\theta' = \sum_{i=1}^{k} \theta_i' / R$, $\text{var}[\theta'] = \sigma^2/R$, need $R \ge \frac{\sigma^2}{\varepsilon^2 \theta^2 \delta}$
2) Median of Means: Take $\theta_1', \dots \theta_k'$ (each is mean of samples of (ε, δ))
$R \ge c \ln(1/\varepsilon)$
Return θ' = median $(\theta_1', \dots, \theta_k')$
If ≥ 0.5 of $\theta_i' \in ((1+\varepsilon)\theta)$, then θ' is (ε, δ) by Chernoff.

Approximate Counting: $f(x) = |x|$, $c(x)$ with size $\log(\log(n))$
Let $c = 0$, each step update $c \to c+1$ w/ prob $\frac{1}{2^c}$

Stream n elements, $f(x) = \#$ distinct elements
pick hash $h: U \to [0,1]$ for $y_i \in [0,1]$
let $c(x_1, \dots, x_k) = \min h(x_i)$
Return $f'(x) = \frac{1}{c(x)} - 1$ since $\mathbb{E}[\min(y_1, \dots, y_d)] = d$

Convolutions & FFT

Roots of unity $\Omega(n) = \{e^{2\pi i k/n} \mid k = 0, \dots, n-1\}$
primitive roots of unity: $\omega = \omega_n = e^{2\pi i/n}$, $\Omega_n^2 = \Omega_{n/2}$

$p(x) = a_0 + a_1 x + \dots + a_{n-1} x^{n-1}$
$q(x) = b_0 + b_1 x + \dots + b_{n-1} x^{n-1}$
} want to multiply these 2 polynomials efficiently.

2 Representations:
- coefficient vector rep: $(a_0, \dots, a_{n-1})$
- point value rep: $(p(x_0), p(x_1), \dots, p(x_{n-1}))$

$V(x) = p(x)q(x) = c_0 + c_1 x_1 + \dots + c_{2(n-1)} x^{2(n-1)}$
where $c_k = \sum_{i+j=k} a_i b_j = \sum_{i=0}^{k} a_i b_{k-i}$

DFT: $(a_0, \dots, a_{n-1}) \to (p(1), p(\omega), \dots, p(\omega^{n-1}))$
FFT: fast algo. for DFT ($O(n \log n)$)
note: n denotes highest degree

Minkowski Sum: 2 sets $A, B \subseteq \mathbb{R}^d$ $A+B = \{x+y : x \in A, y \in B\}$
ex. $A = \{0,1,2\}$ $B = \{0,3,6\}$
0 1 2 → 0 1 2
6 3 0 → 6 3 0
$A+B = \{0,1,3,6,4,2,5,7,8\}$

$x = \{0,1,3\}$ $y = \{2,3,4\}$
$x = x^0 + x^1 + x^3$, $y = x^2 + x^3 + x^4$
$xy = x^k$ if $\exists x^k \in x$, $x^{k-i} \in y$

Substring: use convolution to get $O((m+n)\log(m+n))$ - algo for checking if $p = (p_0, \dots, p_m) \in \{0,1\}^{m+1}$ substring of $s = (s_0, \dots, s_n) \in \{0,1\}^{n+1}$
Idea: want $s_j \cdot p_j$ to encode agreement $s_j = p_j$.
$a_j = \begin{cases} 1 & \text{if } s_j = 1 \\ -1 & \text{if } s_j = 0 \end{cases}$ and $b_k = \begin{cases} 1 & \text{if } p_{m-k} = 1 \\ -1 & \text{if } p_{m-k} = 0 \end{cases}$
Then, $(a * b)_\ell = m+1$ IFF $\ell - m \in H$, and there is an $O(n \log n)$-time algo. for H.

ex. $S = 01101$ $p = 011$ $n=5$ $m=3$ $H = \{0,3\}$
a: -1 1 1 -1 1
b_{rev}: -1 1 1
$a * b$: -1 0 3 -1 3 0 -1
3 = perfect agreement.

FFT for computing DFT:

Input $a_0, \dots, a_{n-1}$: Assume n is a power of 2.
If $n = 1$:
  Return $[a_0]$
Build $a_{even} := (a_0, a_2, \dots, a_{n-2})$, $a_{odd} := (a_1, a_3, \dots, a_{n-1})$
Recursively compute $F_{even} = \text{FFT}(a_{even})$, $F_{odd} = \text{FFT}(a_{odd})$
For $k = \{0, 1, \dots, n/2 - 1\}$:
  Set $F[k] = F_{even}[k] + \omega_n^k \cdot F_{odd}[k]$
  Set $F[k + n/2] = F_{even}[k] + \omega_n^{k+n/2} \cdot F_{odd}[k]$
Return F

$p(x), q(x)$ coefficient vector rep $\xrightarrow{O(n^2)}$ $r(x)$ coefficient vector rep
$O(n \log n) \downarrow$ FFT
$p(x), q(x)$ point value rep $\xrightarrow[O(n)]{r(x) = p(x) \cdot q(x)}$ $r(x)$ point-value rep
$O(n \log n) \uparrow$ inverse FFT.